

UNIVERSITY OF HAIFA

Edge-Centric Intrusion Detection: A Critical Data Science Evaluation

Ilay Nadler

Supervisor: Dr. Uri Itai

July 2026

Abstract

This report critically evaluates the methodology and claims presented in “Edge-Detect: Edge-centric Network Intrusion Detection using Deep Neural Network” (Singh et al., 2021). The original authors proposed a lightweight, sliding-window Recurrent Neural Network (RNN) to detect Distributed Denial-of-Service (DDoS) attacks on resource-constrained edge devices, claiming up to 99% accuracy on the UNSW-NB15 dataset. During reproduction, a major discrepancy was discovered: the dataset released for reproduction in the authors’ own repository is not UNSW-NB15, but a specific, highly imbalanced capture from the CICIDS2017 benchmark. This resubmission addresses three methodological issues raised on the prior version of this report: (1) a train/validation/test leakage in which the test set had been reused as the validation set for early stopping; (2) an insufficiently clear distinction between reproducing the paper’s stated methodology and evaluating an adapted version on a different dataset; and (3) an error analysis that reported only aggregate False Positive/False Negative counts rather than investigating which feature values were associated with errors. All three are corrected here. Two experiments are now reported under a leakage-free train/validation/test methodology: a true reproduction attempt on the actual, official UNSW-NB15 partition, and an adaptation study on the CICIDS2017 file the authors’ repository actually ships. On UNSW-NB15, the paper’s $\sim 99\%$ accuracy claim does not replicate (best model MCC 0.892); on the released CICIDS2017 file, both models are near-saturated ($\text{MCC} > 0.998$), an artifact of that file’s narrow, single-afternoon capture rather than genuine model quality. In both experiments, a stateless Random Forest baseline matches or outperforms the proposed sliding-window architecture in inference speed and most predictive metrics, while the proposed model’s disk-footprint advantage proves highly dataset-dependent rather than a fixed benefit.

Contents

0.1	Executive Summary	2
0.2	Summary of the Source	3
0.3	Critical Evaluation	3
0.4	Methodology: Train / Validation / Test Separation	4
0.5	Exploratory Data Analysis	5
0.5.1	Experiment A: UNSW-NB15	5
0.5.2	Experiment B: CICIDS2017	5
0.6	Feature Engineering Analysis	6
0.6.1	Experiment A: UNSW-NB15	6
0.6.2	Experiment B: CICIDS2017	6
0.7	Reproducibility Analysis	6
0.8	Experimental Results	7
0.8.1	Evaluation Metrics	7
0.8.2	Experiment A: UNSW-NB15 Results	8
0.8.3	Experiment B: CICIDS2017 Results	8
0.8.4	Cross-Experiment Comparison	9
0.9	Conclusions and Error Analysis	9
0.9.1	Error Analysis	9
0.9.2	Conclusions	10
0.9.3	Lessons Learned	10
0.10	Summing It Up	10
	Bibliography	12

0.1 Executive Summary

This report critically evaluates “Edge-Detect: Edge-centric Network Intrusion Detection using Deep Neural Network” (Singh et al., 2021), which proposes a lightweight sliding-window recurrent (LSTM/GRU-based) architecture for detecting Distributed Denial-of-Service (DDoS) attacks on resource-constrained edge devices, claiming up to 99% accuracy on the UNSW-NB15 dataset.

The paper states that the authors evaluated their model on the UNSW-NB15 dataset. However, reproducing the work from the authors’ own public repository revealed that the artifact actually shipped for reproduction is not UNSW-NB15 at all: it is the equally well-known CICIDS2017 `Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.csv` file. This discrepancy between the paper’s stated methodology and its released reproducible artifact is undocumented and is, in itself, a central reproducibility finding of this report. The main issue about this is not the gap in reproducibility, but the fact that the modified dataset being used in the implementation has unrealistic prevalence, where the majority of samples are DDoS attacks. Looking at the originally mentioned dataset, it also has attack-heavy prevalence. In this report, along with the Python notebook, we see how the claims hold for both dataset, taking into account the special conditions the authors state as an advantage to their specific method - low-resource environments and quick response times.

This is a resubmission that addresses three methodological issues raised on the prior version: (1) the test set had been reused as the validation set during training and early stopping, so it could not be treated as an independent final test set; (2) the project’s single CICIDS2017-based experiment was not a direct reproduction of the paper’s UNSW-NB15 evaluation, and this was not distinguished clearly enough in the writeup. Added to the notebook is also a reproduction with the intended dataset. (3) the error analysis reported only aggregate False Positive/False Negative counts rather than investigating which samples and feature values were actually associated with errors. All three are addressed below and reflected in the accompanying notebook.

Methodology fix: every experiment in this report now uses an independent three-way train/validation/test split. The validation split – not the test split – is used for early stopping. The test split is touched exactly once, for final evaluation.

Reproduction vs. adaptation: the project now reports **two** clearly separated experiments. **Experiment A** is a true reproduction attempt using the actual, official UNSW-NB15 training/testing partition – the dataset the paper claims to use. **Experiment B** is an adaptation study using the CICIDS2017 file the authors’ own repository actually ships (documented in the Reproducibility Analysis), kept explicitly separate from Experiment A rather than conflated with it.

Key Findings: On the true UNSW-NB15 reproduction (Experiment A), neither model comes close to the paper’s claimed 99% accuracy figure once evaluated with a properly independent test set: the Random Forest baseline reaches MCC 0.892 and the GRU reaches MCC 0.832, both well short of saturation. On the CICIDS2017 adaptation (Experiment B), both models are far closer to ceiling (MCC > 0.998) even after fixing the train/validation/test leakage – confirming that the near-perfect scores reported previously were a property of this specific, narrow attack-day capture, not an artifact of the leakage itself. In both experiments, the Random Forest baseline matches or exceeds the proposed sliding-window architecture on predictive metrics while being an order of magnitude faster at inference, though the GRU retains a smaller disk footprint in both cases.

0.2 Summary of the Source

The paper “Edge-Detect: Edge-centric Network Intrusion Detection using Deep Neural Network” by Singh et al. tackles Network Intrusion Detection in the specific domain of edge-centric networks.

An *intrusion detection system* (IDS) is an application that sits on top of a network or some system to monitor malicious activity [5]. It may be a central computer that collects data from all nodes of a network to be aware of all of it at once, or it may sit at critical sections and monitor locally. It differs from a firewall by the way it detects anomalies: a firewall operates by preset, static rules for preventing suspicious activity from entering the network, while an IDS monitors activity inside the network and alerts on potential maliciousness. By its nature, it is more dynamic.

In particularly large networks, IDS is highly influential compared to more traditional security measures like firewalls, because:

- Networks are distributed and data cannot move from all nodes to a central computer in a reasonable time.
- Activities interpreted as threats in one part of the network can be normal in another.
- The network being a dynamic structure requires highly dynamic security measures.

The paper tackles the challenge of fast and dynamic, distributed IDS. It proposes a framework to detect DDoS attacks composed of many lightweight Deep Learning (DL) models that sit on the edges between nodes of the network. It trains sequential learning models such as RNN, LSTM, and GRU to do so in little time and with few computational resources.

Stated dataset (per the paper): The authors state that they selected the UNSW-NB15 dataset for their evaluation [1], citing its relative modernity, clean training/testing separation, and correctly labelled features. UNSW-NB15 has 49 native fields [3], including protocol (proto), service (service), connection state (state), and byte/packet-level flow statistics.

Actual dataset released for reproduction: As detailed in the Reproducibility Analysis, the authors’ own repository does not contain UNSW-NB15. It contains `Friday-WorkingHours-Afternoon-DDos`, a well-known file from the CICIDS2017 benchmark [4], with 78 CICFlowMeter-derived features plus a binary label and 225,745 rows.

Because of this mismatch, this report evaluates **both** datasets separately rather than picking one: Experiment A uses the genuine, official UNSW-NB15 partition to test the paper’s claim as stated; Experiment B uses the CICIDS2017 file the authors’ code actually runs against, to test what a practitioner following the released repository would actually reproduce.

0.3 Critical Evaluation

The core claim made by the authors is that their proposed recurrent pipeline can achieve highly reliable classification accuracy (up to 99%) while retaining a model size small enough to execute on resource-constrained devices like a Raspberry Pi. Note that neither this report nor the original paper’s own evaluation actually deploys either model on such hardware; disk footprint and inference latency (Section 0.8) are used as proxies, measured on the training machine rather than a physically constrained device (see the limitation noted in Summing It Up).

The dataset claim does not match the released artifact: before evaluating the accuracy claim itself, a fundamental issue emerged during reproduction. The paper states the authors used UNSW-NB15; the authors’ own repository does not contain it. A reader attempting to verify the paper’s central claim using the authors’ released code and data would, in practice, be testing it against CICIDS2017 instead, with no note in the repository explaining the substitution. This is why the present report treats reproducing the paper’s stated methodology (Experiment A, UNSW-NB15) and evaluating the released artifact (Experiment B, CICIDS2017) as two separate exercises rather than one.

Evaluating the accuracy claim on its own stated dataset (Experiment A): with a properly independent test set, the paper’s $\sim 99\%$ accuracy claim is not supported. The Random Forest baseline reaches an MCC of 0.892 and 95.0% accuracy; the GRU reproduction reaches an MCC of 0.832 and 92.2% accuracy (Section 0.8). Neither approaches saturation, and the sliding-window GRU is the weaker of the two models on every predictive metric measured. Moreover, even the official UNSW-NB15 partition is itself attack-majority (63.9% attack in both the training and test splits used here) – so a model trained on it is not calibrated for the benign-heavy traffic a real edge network would see, independent of any dataset-substitution issue.

Evaluating the released artifact (Experiment B): the CICIDS2017 file actually shipped is an isolated capture of a single Friday afternoon during which a DDoS exercise was specifically staged, so its 56.7% DDoS composition reflects the choice of capture window, not a property of network traffic in general. Both models are close to saturated on this file ($\text{MCC} > 0.998$), which is a property of the file’s homogeneity rather than evidence that either model generalizes well.

Evaluation of Author’s Methodology: taken together, the two experiments show that the paper’s evaluation setup does not transfer to a realistic edge deployment on either dataset: on UNSW-NB15 the claimed accuracy does not replicate under proper evaluation methodology, and on the released CICIDS2017 file the near-perfect scores reflect an unrepresentative capture window rather than genuine generalization. In both cases, the authors’ architectural claim that a complex sliding-window RNN ($T = 10$) is necessary to capture temporal attack states is contradicted by evidence: a stateless Random Forest baseline matched or outperformed it in both experiments, indicating the computationally expensive sequential transformation is not earning its cost.

0.4 Methodology: Train / Validation / Test Separation

The prior submission trained the GRU with `validation_data` set to the test dataset, and used `EarlyStopping/ModelCheckpoint` monitoring `val_loss/val_auc` against it. Because the model’s stopping point and best-weights checkpoint were selected using performance on that set, it had already influenced the trained model before “final” evaluation, so it no longer functioned as an independent test set.

Both experiments in this resubmission use an explicit three-way, stratified split (70% / 15% / 15% train / validation / test in both experiments). The validation split is used for early stopping (`EarlyStopping(monitor='val_loss', patience=3, restore_best_weights=True)`); the test split is not passed to `model.fit()` at any point and is used only once, after training is complete, for the metrics reported in Section 0.8. The scaler (`StandardScaler`) is also fit on the training split only, and applied to validation and test without refitting, so no distributional in-

formation from either held-out split leaks into preprocessing. The Random Forest baseline does not require a validation set for early stopping, so it is trained on the combined training + validation data and evaluated on the identical held-out test split used for the GRU, keeping the comparison between the two models fair.

One useful side-effect of the fix: re-running Experiment B (CICIDS2017) with proper separation still yields near-perfect scores (MCC 0.9989 for the GRU, 0.9999 for the Random Forest). This confirms that the previous submission’s near-ceiling results were not primarily an artifact of the validation leakage – they are a property of the CICIDS2017 file’s homogeneity (Section 0.8). The leakage was a genuine methodological error regardless, since it meant the reported test metrics could not be trusted as independent, even though in this particular case the underlying numbers happened not to move much.

0.5 Exploratory Data Analysis

0.5.1 Experiment A: UNSW-NB15

Distributions and outliers: numeric flow features (byte counts, packet counts, inter-arrival times) are heavily right-skewed with long tails consistent with burst traffic during an attack. Outliers are retained rather than clipped, since they frequently carry the attack signal itself.

Missing values: no missing or infinite values were found in the official UNSW-NB15 partition after loading; no rows were dropped on this basis (the flow-rate division-by-zero issue found in Experiment B does not occur here, since UNSW-NB15’s feature set is computed differently).

Columns and index sanity check: no duplicate column names, constant columns, or exact duplicate-value columns were found among UNSW-NB15’s native fields.

Class imbalance (prevalence): the combined official train+test partition is 63.9% attack / 36.1% normal. This is the dataset the paper claims to use, and it is itself far from the benign-heavy composition of a real edge network – a finding independent of the CICIDS2017 substitution issue.

Correlation analysis: Spearman correlation (chosen for the same reasons as in Experiment B, see below) with a threshold of $|\rho| > 0.9$ identified 22 redundant numeric features (e.g. dbytes, sload, dload, sjit, djit, tcprtt), leaving 21 numeric features plus the genuinely categorical proto, service, and state fields.

0.5.2 Experiment B: CICIDS2017

Distributions and outliers: the numerical flow-based features (e.g., Flow Duration, Flow Packets/s) are heavily right-skewed, with a long tail of extreme values consistent with burst traffic during an attack. These outliers are retained rather than clipped.

Missing values: a leading ID_Column mismatch between the shipped header file and the data file first had to be corrected: the header file lists one fewer name than the data has columns. Once aligned, Flow Bytes/s contained 4 conventional missing values; several flow-rate columns additionally contained Infinity values from division-by-zero when flow duration is 0. Both were dropped, reducing the dataset from 225,745 to 225,711 rows.

Columns and index sanity check: the header file lists Fwd Header Length twice; the duplicate is renamed before any per-column operation. 17 columns are exactly constant or exact duplicate-value columns (e.g. Bwd PSH Flags, CWE Flag Count, all six bulk-rate columns,

and the subflow columns that exactly duplicate their non-subflow counterparts) and are dropped outright.

Class imbalance (prevalence): 56.72% DDoS / 43.28% Benign. This reflects the choice to release an isolated attack-day capture rather than a cross-section of ordinary edge traffic.

Correlation analysis: Spearman correlation was chosen over Pearson because several flow-based features (e.g., `Flow Packets/s` versus `Flow Bytes/s`) are monotonically but non-linearly related, and Spearman’s rank-based formulation is more robust to the heavy-tailed outliers identified above than Pearson’s linear assumption, without the smaller-sample correction that would favor Kendall’s τ . Pairs with $|\rho| > 0.9$ identified 38 additional redundant features.

0.6 Feature Engineering Analysis

0.6.1 Experiment A: UNSW-NB15

Unlike the CICIDS2017 file, UNSW-NB15 contains genuine string-valued categorical columns: `proto` (e.g. `tcp`, `udp`, `arp`, `unas`, and dozens of rarer protocols), `service` (e.g. `http`, `ftp`, `dns`, `radius`, `-`), and `state` (e.g. `FIN`, `CON`, `INT`, `REQ`). One-Hot Encoding is therefore actually exercised here, unlike in Experiment B: 21 numeric features plus 3 categorical columns expand to 171 features after encoding, since `proto` and `service` each take on dozens of rare values. Standard scaling (Z-score normalization) is applied after encoding to keep large-magnitude byte/packet features from dominating gradient descent.

The 22 Spearman-redundant numeric features identified in the EDA are dropped before encoding. This reduces the pre-encoding feature count and, more importantly for an edge deployment, avoids propagating redundant signal into the one-hot expansion.

Additional features that could improve performance: rolling, multi-flow statistics – e.g. the rate of distinct destination IPs contacted by a source over a short window, or the ratio of SYN to completed-handshake connections – would give the model more direct evidence of a DDoS ramp-up than per-flow features alone, since DDoS is fundamentally a multi-flow, volumetric phenomenon.

0.6.2 Experiment B: CICIDS2017

This CICIDS2017 file contains no string-valued categorical columns: all 78 `CICFlowMeter` features are already numeric, so One-Hot Encoding logic is retained in the pipeline for parity with Experiment A but finds nothing to encode. Standard scaling is applied for the same gradient-descent reasons as in Experiment A.

The 38 Spearman-redundant features plus the 17 exact constant/duplicate-value features identified in the EDA (55 total, with some overlap) are dropped, leaving 31 features before training. A crucial hidden preprocessing step involved addressing the division-by-zero-induced `Infinity` values in flow-rate features described in the EDA; the affected rows were dropped rather than imputed, since imputing a rate over a zero-duration flow has no clear interpretation.

0.7 Reproducibility Analysis

The official implementation for the “Edge-Detect” paper is hosted on GitHub under <https://github.com/racsa-lab/Edge-Detect> [2].

Dataset mismatch: the paper’s text states the authors used the UNSW2015 dataset. The repository’s `dataset/` folder, however, contains exactly three files: the raw CICIDS2017 file `Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.csv`, a header file (`ddos_column_headers`), and a pre-processed `modified_dataset.csv` that is byte-for-byte identical (verified by hash) to the one used in Experiment B. There is no UNSW-NB15 file anywhere in the repository, and the README does not mention or explain the substitution.

Execution and Dependencies: for Experiment A, the official, publicly released UNSW-NB15 training/testing partition (Moustafa & Slay, 2015) was obtained directly, since this is not available from the authors’ own repository. For Experiment B, the exact files released by the authors were used as-is. Both pipelines executed successfully in a Python/TensorFlow environment.

Hidden Preprocessing: loading Experiment B’s released files revealed a header-alignment bug (`ddos_column_headers` lists one fewer column name than `modified_dataset.csv` has), requiring a manually inserted leading `ID.Column` name to align the two – exactly the kind of silent, undocumented preprocessing step the assignment brief warns can break naive reproduction. No comparable issue was found in the official UNSW-NB15 files used for Experiment A.

Overall reproducibility: the paper’s architecture is straightforward to reconstruct from its description, but claim verification is compromised by the dataset substitution: without independently sourcing UNSW-NB15 ourselves (Experiment A), a reader following only the authors’ repository would silently reproduce results on a materially different, easier-natured dataset (Experiment B) without any indication that this had happened.

0.8 Experimental Results

Two models are trained and evaluated in each experiment: the Edge-Detect reproduction (sliding-window GRU, $T = 10$) and a stateless Random Forest baseline, using the train/validation/test methodology described above.

0.8.1 Evaluation Metrics

As in the prior submission, Accuracy, Precision, and Recall are computed but are not the primary basis for comparison, since in Experiment B they saturate near 1.00 for both models and are uninformative at that ceiling. ROC-AUC is computed but similarly omitted from the discussion for the same reason in Experiment B. MCC and the confusion matrix are used as the primary comparison metrics in both experiments, since MCC remains informative even under class imbalance and near-saturation, and the confusion matrix is the basis for the deeper error analysis in the next section.

0.8.2 Experiment A: UNSW-NB15 Results

	GRU (Edge-Detect, $T=10$)		Random Forest	
Accuracy	0.9207		0.9509	
Precision (macro)	0.9137		0.9460	
Recall (macro)	0.9184		0.9463	
MCC	0.8315		0.8941	
Disk footprint	0.26 MB		187.5 MB	
Latency	0.382 ms/window		0.036 ms/packet	

	GRU		Random Forest	
	Pred. Attack	Pred. Normal	Pred. Attack	Pred. Normal
Actual Attack	TP = 23068	FN = 1629	TP = 23749	FN = 952
Actual Normal	FP = 1436	TN = 12509	FP = 945	TN = 13005

Note on sample sizes: Across all evaluations, the Random Forest baseline evaluates exactly 9 more test samples than the GRU. This is not a data split error; it is the mathematically expected result of the dataset transformation. The GRU utilizes a sliding window of size $T = 10$, which inherently consumes the first 9 time-steps to form the initial sequence, resulting in $(N - T + 1)$ evaluable windows.

The Random Forest outperforms the GRU on every predictive metric while using a dramatically larger amount of disk space (187.5 MB vs. 0.26 MB) – a genuine trade-off, unlike in Experiment B where the RF’s footprint was modest. This is a direct consequence of the 171-feature one-hot-encoded input space: default-depth Random Forest trees grow large when the input has many sparse binary columns. The GRU remains dramatically faster to store but slower per inference than the Random Forest.

0.8.3 Experiment B: CICIDS2017 Results

	GRU (Edge-Detect, $T=10$)		Random Forest	
Accuracy	0.9996		1.0000	
Precision (macro)	0.9996		1.0000	
Recall (macro)	0.9996		1.0000	
MCC	0.9991		0.9999	
Disk footprint	0.108 MB		1.03 MB	
Latency	0.334 ms/window		0.0065 ms/packet	

	GRU		Random Forest	
	Pred. Attack	Pred. Normal	Pred. Attack	Pred. Normal
Actual Attack	TP = 19192	FN = 8	TP = 19203	FN = 1
Actual Normal	FP = 7	TN = 14641	FP = 0	TN = 14653

With proper train/validation/test separation, results are essentially unchanged from the previous (leaked) submission: both models remain near-saturated. The Random Forest makes a single error across 33,857 test samples.

0.8.4 Cross-Experiment Comparison

	Experiment A (UNSW-NB15)	Experiment B (CICIDS2017)
Test-set attack prevalence	63.9%	56.7%
GRU MCC	0.832	0.999
RF MCC	0.892	1.000

The gap between the two experiments is itself informative: the paper’s claimed accuracy does not replicate on the dataset it claims to use, while the near-perfect scores obtained on the substituted CICIDS2017 file are specific to that file’s narrow, homogeneous capture window rather than evidence of genuine model quality.

0.9 Conclusions and Error Analysis

0.9.1 Error Analysis

In intrusion detection, a False Negative allows a payload to bypass the IDS, potentially causing complete network downtime; excessive False Positives overwhelm administrators with alert fatigue. Beyond these aggregate counts, this resubmission investigates which specific feature values are associated with the Random Forest’s errors in each experiment.

Experiment A (UNSW-NB15): with 1,897 misclassified samples out of 38,651 (945 FP + 952 FN), aggregate comparison between misclassified and correctly classified samples is meaningful. The strongest signal is `dttl` (destination time-to-live): its mean among misclassified samples is 180.6, versus 78.7 among correctly classified samples – more than double. This suggests the model struggles specifically with traffic whose destination TTL falls outside the range the bulk of the training distribution occupies, consistent with these being edge cases the model has not seen enough of, rather than errors spread uniformly across the traffic population. Certain rare `service` categories (`ftp`, `radius`) are also over-represented among errors relative to their base rate, though several other apparent signals in the raw comparison are one-hot dummy columns whose apparent 100% relative difference is an artifact of their rarity in the error subset rather than a genuine pattern, and are not interpreted as such.

Experiment B (CICIDS2017): with only a single residual error (1 False Negative, 0 False Positives) out of 33,857 test samples, aggregate feature-distribution comparisons are not statistically meaningful at this sample size. The one misclassified sample was inspected individually: it had an unusually high `Destination Port` value (in the high-numbered ephemeral range) relative to the typical destination ports seen among correctly classified flows, suggesting the model’s one failure mode on this dataset involves atypical port usage rather than a systematic weakness. Given the near-total absence of errors, this is best read as a description of this specific model’s blind spot on this specific file rather than a generalizable pattern – a further indication that this dataset’s near-ceiling performance reflects task ease rather than robust generalization.

Common implication: because Experiment B’s released dataset carries an unrealistic attack-heavy prevalence, a model’s decision threshold calibrated on it would be biased toward predicting an attack if deployed against realistic, benign-heavy edge traffic, producing a cascade of False Positives beyond what either experiment’s test scores suggest. The same caution applies to Experiment A, whose official partition is similarly attack-majority.

0.9.2 Conclusions

The most significant finding of this reproduction is that the paper’s stated dataset (UNSW-NB15) and its released reproducible artifact (CICIDS2017) do not match, and that this matters in practice: the paper’s claimed accuracy does not replicate under a properly independent test evaluation on the dataset it actually claims to use (Experiment A, MCC 0.83–0.89), while the near-perfect scores obtainable on the substituted file (Experiment B, $\text{MCC} > 0.998$) are an artifact of that file’s narrow scope rather than genuine evidence for the paper’s claims.

Separately, in both experiments the original authors’ sliding-window RNN architecture is outperformed or matched by a stateless Random Forest on every predictive metric, while being slower at inference in both cases. The GRU’s smaller disk footprint is a genuine advantage in Experiment B (0.11 MB vs. 1.03 MB) but is a much larger, likely decisive advantage in Experiment A (0.26 MB vs. 187.5 MB, since one-hot encoding UNSW-NB15’s categorical fields makes the Random Forest far larger). The overall trade-off is dataset-dependent: for a feature space that produces a compact Random Forest, the GRU’s added computational latency is a poor trade for a bigger model; for a feature space that produces a very large Random Forest (many sparse one-hot columns, as in Experiment A), the GRU’s disk-footprint advantage becomes more compelling despite its own latency and accuracy costs.

0.9.3 Lessons Learned

- An independent validation set is necessary whenever early stopping or model checkpointing is used; reusing the test set for this purpose silently invalidates the final evaluation even when, as in Experiment B, the reported numbers do not move much once corrected.
- Verifying a paper’s claimed dataset against what its released reproduction artifact actually contains is a distinct and necessary reproducibility check, separate from checking whether the code executes.
- Aggregate FP/FN counts can mask both genuinely informative error patterns (as in Experiment A’s `dt1` finding) and situations where the error count is too small for aggregate analysis to be meaningful at all (as in Experiment B).

0.10 Summing It Up

This project evaluated “Edge-Detect: Edge-centric Network Intrusion Detection using Deep Neural Network,” which proposes a lightweight sliding-window recurrent architecture for DDoS detection on edge devices, claiming up to 99% accuracy on UNSW-NB15.

Reproducing this work surfaced a dataset mismatch: the paper’s stated dataset (UNSW-NB15) does not match its released reproducible artifact (a CICIDS2017 DDoS capture), undocumented in the repository. Rather than choose one dataset, this report evaluates both, using a methodologically corrected (leakage-free) train/validation/test split throughout.

On the paper’s actually-claimed dataset (Experiment A, UNSW-NB15), the $\sim 99\%$ accuracy claim does not replicate: the best model (Random Forest) reaches MCC 0.892, and the proposed GRU reaches MCC 0.832, both meaningfully short of saturation. On the dataset actually released for reproduction (Experiment B, CICIDS2017), both models are near-perfect ($\text{MCC} > 0.998$), but this reflects the narrow, single-afternoon nature of that capture rather than

genuine model quality; a single-sample error analysis on this experiment further underscores how little there is to learn from its near-zero error rate. In both experiments, the stateless Random Forest baseline matches or outperforms the proposed sliding-window architecture on every predictive metric while offering dramatically faster inference; the GRU’s compactness is a genuine, and in Experiment A a substantial, advantage, but its predictive and latency costs are not clearly justified by that advantage alone in either case.

Deploying this specific implementation in a real-world edge scenario is **not recommended** without: resolving and disclosing the dataset discrepancy; evaluating on traffic with realistic (benign-majority) prevalence rather than either benchmark’s attack-heavy partition; and reconsidering whether the sliding-window architecture’s latency and accuracy costs are justified by its disk-footprint advantage, which this report finds to be highly dataset-dependent rather than a fixed benefit.

Scope limitation: neither this report nor, as far as could be determined, the original paper actually deploys or benchmarks either model on constrained edge hardware (e.g. a Raspberry Pi). Disk footprint (MB) and inference latency (ms/window or ms/packet) reported throughout Section 0.8 were measured on the same machine used for training, as proxies for edge feasibility, not on a physically constrained device. Actual edge deployment introduces further constraints – limited RAM and CPU, no GPU, power budget, thermal throttling – that these proxies do not capture, and which could change the balance between the two models beyond what is reported here. A genuine test of the paper’s central claim would require running both models on the target hardware class itself.

Bibliography

- [1] P. Singh, J. J. P. A. Pankaj, and R. Mitra, “Edge-Detect: Edge-Centric Network Intrusion Detection using Deep Neural Network,” in *2021 IEEE 18th Annual Consumer Communications & Networking Conference (CCNC)*, pp. 1–6, 2021. <https://doi.org/10.1109/CCNC49032.2021.9369469>
- [2] racsa-lab, “Edge-Detect” [GitHub repository], <https://github.com/racsa-lab/Edge-Detect>
- [3] N. Moustafa and J. Slay, “UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set),” in *2015 Military Communications and Information Systems Conference (MilCIS)*, pp. 1–6, 2015.
- [4] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, “Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization,” in *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, pp. 108–116, 2018.
- [5] Wikipedia contributors, “Intrusion detection system,” *Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/wiki/Intrusion_detection_system